These two APIs correspond to the paths `/` and `/person`, respectively. This basic functionality is useful, but typically you need some of your APIs to be more dynamic. It's unlikely, for example, that the `/person` API would be useful in practice, as it always returns the same `Person` object. What might be more useful is an API to which you can provide the user's first name, and the API returns all the users with that name.

You can achieve this goal by using *parameterized routes* for your API definitions. You can create a parameter in a minimal API route using the expression `{someValue}`, where `someValue` is any name you choose. The value will be extracted from the request URL's path and can be used in the lambda function endpoint.

**NOTE** I introduce only the basics of extracting values from routes in this chapter. You'll learn a lot more about routing in chapter 6, including why we use routing and how it fits into the ASP.NET Core pipeline, as well as the syntax you can use.

If you create an API using the route template `/person/{name}`, for example, and send a request to the path `/person/Andrew`, the `name` parameter will have the value `"Andrew"`. You can use this feature to build more useful APIs, such as the one shown in the following listing.

**Listing 5.1 A minimal API that uses a value from the URL**

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);
WebApplication app = builder.Build();

var people = new List<Person>                                    ❶
{                                                                ❶
    new("Tom", "Hanks"),                                         ❶
    new("Denzel", "Washington"),                                 ❶
    new("Leondardo", "DiCaprio"),                                ❶
    new("Al", "Pacino"),                                         ❶
```